

```

from keras.layers import Input, Dense
from keras.models import Model
from keras.datasets import mnist, fashion_mnist
import numpy as np
import matplotlib.pyplot as plt

# Modified autoencoder with an additional hidden layer
encoding_dim = 32 # Size of encoded representation

# Define the input placeholder (for 28x28 images flattened to 784)
input_img = Input(shape=(784,))

# Encoder
encoded = Dense(128, activation='relu')(input_img) # First hidden
layer
encoded = Dense(64, activation='relu')(encoded) # Additional
hidden layer
encoded = Dense(32, activation='relu')(encoded) # Bottleneck layer

# Decoder
decoded = Dense(64, activation='relu')(encoded) # Expanding from
bottleneck
decoded = Dense(128, activation='relu')(decoded)
decoded = Dense(784, activation='sigmoid')(decoded) # Reconstructing
the original input

# Model
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# Load the Fashion MNIST dataset
(x_train, _), (x_test, _) = fashion_mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))

# Train autoencoder
history = autoencoder.fit(x_train, x_train,
                          epochs=50,
                          batch_size=256,
                          shuffle=True,
                          validation_data=(x_test, x_test))

Epoch 1/50
235/235 ————— 4s 8ms/step - loss: 0.4619 - val_loss:
0.3184
Epoch 2/50
235/235 ————— 1s 3ms/step - loss: 0.3130 - val_loss:
0.3074

```

```
Epoch 3/50
235/235 _____ 1s 3ms/step - loss: 0.3036 - val_loss:
0.3012
Epoch 4/50
235/235 _____ 1s 3ms/step - loss: 0.2978 - val_loss:
0.2976
Epoch 5/50
235/235 _____ 1s 3ms/step - loss: 0.2956 - val_loss:
0.2950
Epoch 6/50
235/235 _____ 1s 4ms/step - loss: 0.2919 - val_loss:
0.2928
Epoch 7/50
235/235 _____ 1s 4ms/step - loss: 0.2902 - val_loss:
0.2916
Epoch 8/50
235/235 _____ 1s 3ms/step - loss: 0.2888 - val_loss:
0.2900
Epoch 9/50
235/235 _____ 1s 3ms/step - loss: 0.2871 - val_loss:
0.2887
Epoch 10/50
235/235 _____ 1s 3ms/step - loss: 0.2864 - val_loss:
0.2873
Epoch 11/50
235/235 _____ 1s 3ms/step - loss: 0.2849 - val_loss:
0.2866
Epoch 12/50
235/235 _____ 1s 3ms/step - loss: 0.2840 - val_loss:
0.2857
Epoch 13/50
235/235 _____ 1s 3ms/step - loss: 0.2833 - val_loss:
0.2853
Epoch 14/50
235/235 _____ 1s 3ms/step - loss: 0.2826 - val_loss:
0.2845
Epoch 15/50
235/235 _____ 1s 3ms/step - loss: 0.2815 - val_loss:
0.2841
Epoch 16/50
235/235 _____ 1s 3ms/step - loss: 0.2815 - val_loss:
0.2837
Epoch 17/50
235/235 _____ 1s 3ms/step - loss: 0.2810 - val_loss:
0.2830
Epoch 18/50
235/235 _____ 1s 3ms/step - loss: 0.2808 - val_loss:
0.2826
Epoch 19/50
```

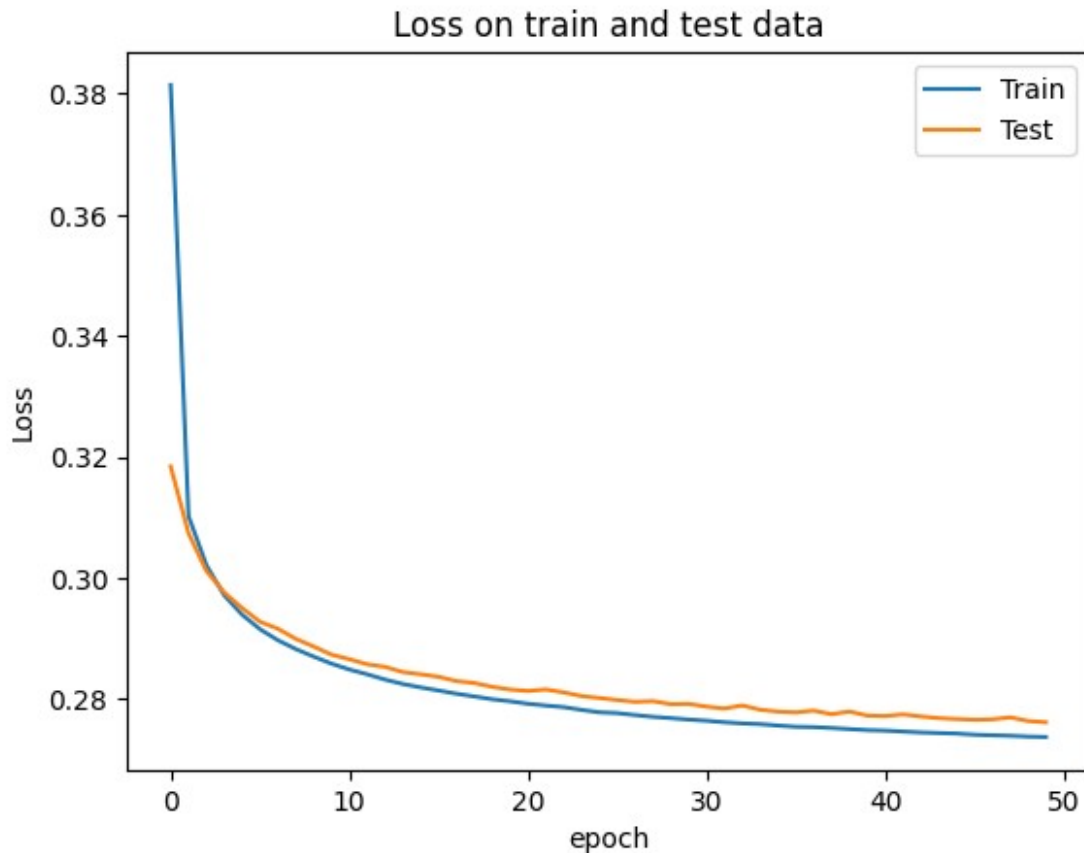
```
235/235 ————— 1s 3ms/step - loss: 0.2796 - val_loss: 0.2820
Epoch 20/50
235/235 ————— 1s 4ms/step - loss: 0.2802 - val_loss: 0.2816
Epoch 21/50
235/235 ————— 1s 4ms/step - loss: 0.2795 - val_loss: 0.2813
Epoch 22/50
235/235 ————— 1s 3ms/step - loss: 0.2787 - val_loss: 0.2816
Epoch 23/50
235/235 ————— 1s 3ms/step - loss: 0.2789 - val_loss: 0.2811
Epoch 24/50
235/235 ————— 1s 3ms/step - loss: 0.2783 - val_loss: 0.2805
Epoch 25/50
235/235 ————— 1s 3ms/step - loss: 0.2785 - val_loss: 0.2802
Epoch 26/50
235/235 ————— 1s 3ms/step - loss: 0.2779 - val_loss: 0.2798
Epoch 27/50
235/235 ————— 1s 3ms/step - loss: 0.2772 - val_loss: 0.2795
Epoch 28/50
235/235 ————— 1s 3ms/step - loss: 0.2777 - val_loss: 0.2796
Epoch 29/50
235/235 ————— 1s 3ms/step - loss: 0.2766 - val_loss: 0.2791
Epoch 30/50
235/235 ————— 1s 3ms/step - loss: 0.2760 - val_loss: 0.2792
Epoch 31/50
235/235 ————— 1s 4ms/step - loss: 0.2773 - val_loss: 0.2787
Epoch 32/50
235/235 ————— 1s 4ms/step - loss: 0.2764 - val_loss: 0.2784
Epoch 33/50
235/235 ————— 1s 3ms/step - loss: 0.2760 - val_loss: 0.2789
Epoch 34/50
235/235 ————— 1s 3ms/step - loss: 0.2758 - val_loss: 0.2782
Epoch 35/50
235/235 ————— 1s 3ms/step - loss: 0.2755 - val_loss:
```

```
0.2779
Epoch 36/50
235/235 ————— 1s 3ms/step - loss: 0.2761 - val_loss:
0.2778
Epoch 37/50
235/235 ————— 1s 3ms/step - loss: 0.2755 - val_loss:
0.2781
Epoch 38/50
235/235 ————— 1s 3ms/step - loss: 0.2751 - val_loss:
0.2775
Epoch 39/50
235/235 ————— 1s 3ms/step - loss: 0.2746 - val_loss:
0.2779
Epoch 40/50
235/235 ————— 1s 3ms/step - loss: 0.2753 - val_loss:
0.2773
Epoch 41/50
235/235 ————— 1s 3ms/step - loss: 0.2752 - val_loss:
0.2772
Epoch 42/50
235/235 ————— 1s 3ms/step - loss: 0.2747 - val_loss:
0.2775
Epoch 43/50
235/235 ————— 1s 3ms/step - loss: 0.2744 - val_loss:
0.2771
Epoch 44/50
235/235 ————— 1s 3ms/step - loss: 0.2745 - val_loss:
0.2768
Epoch 45/50
235/235 ————— 1s 4ms/step - loss: 0.2741 - val_loss:
0.2767
Epoch 46/50
235/235 ————— 1s 4ms/step - loss: 0.2744 - val_loss:
0.2766
Epoch 47/50
235/235 ————— 1s 3ms/step - loss: 0.2733 - val_loss:
0.2766
Epoch 48/50
235/235 ————— 1s 3ms/step - loss: 0.2737 - val_loss:
0.2770
Epoch 49/50
235/235 ————— 1s 3ms/step - loss: 0.2745 - val_loss:
0.2764
Epoch 50/50
235/235 ————— 1s 3ms/step - loss: 0.2730 - val_loss:
0.2762
```

```
# plot loss and accuracy using history object
```

```
plt.plot(autoencoder.history.history['loss'])
```

```
plt.plot(autoencoder.history.history['val_loss'])
plt.xlabel("epoch")
plt.ylabel("Loss")
plt.title("Loss on train and test data")
plt.legend(['Train', 'Test'], loc='upper right')
plt.show()
```



```
# plot the test image
def validate_image(data):

    random_sample = np.random.choice(data.shape[0],1,replace = False)
    print(f"Testing on image {random_sample[0]} \n")

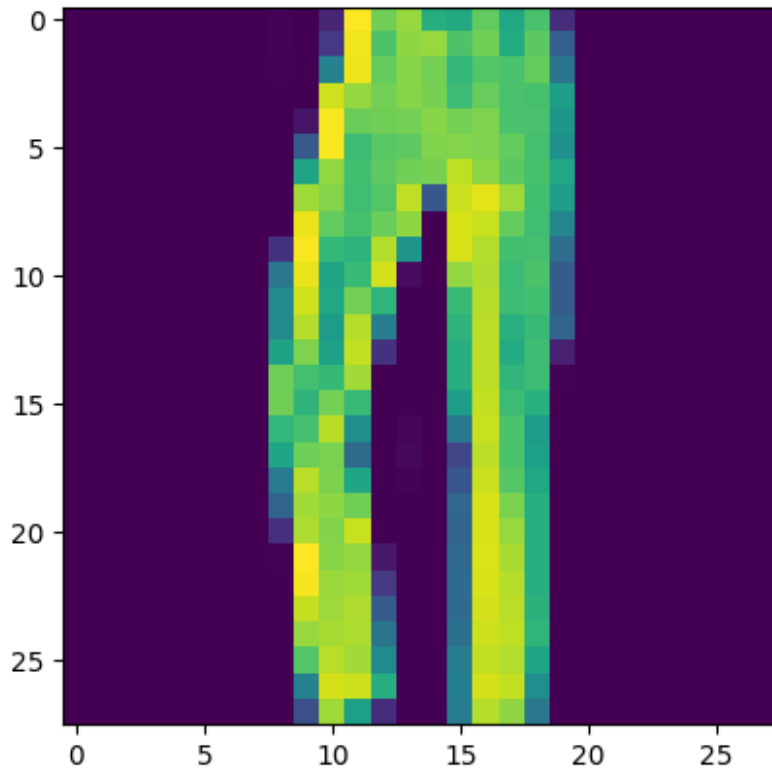
    # plot a sample of test data
    plt.imshow(data[random_sample].reshape(28,28))
    plt.show()

    # plot the sample using reconstructed test data

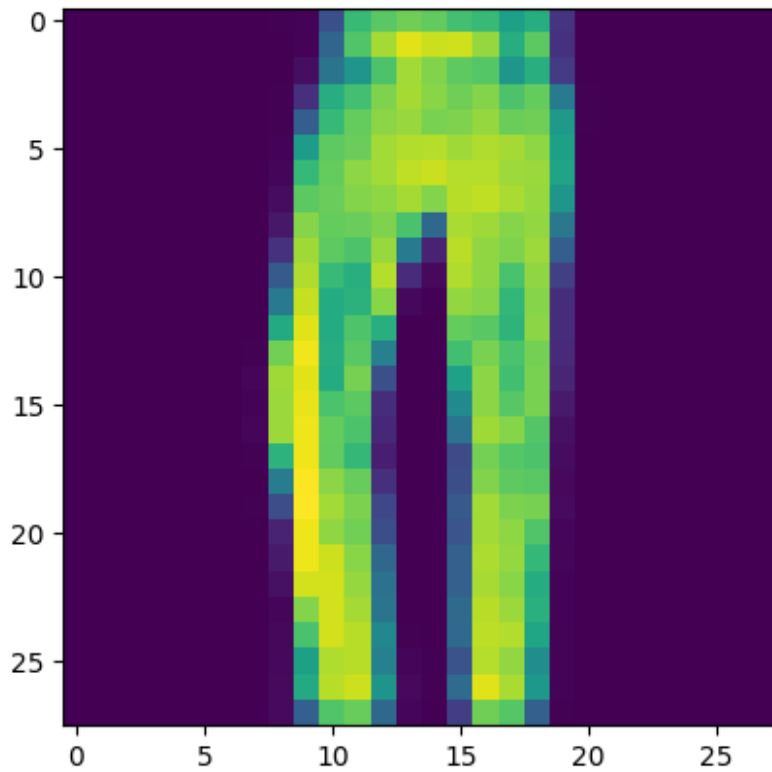
    pred = autoencoder.predict(data[random_sample].reshape(-1,784))
    plt.imshow(pred.reshape(28,28))
    plt.show()
```

```
validate_image(x_test)
```

Testing on image 4648



1/1 ————— 0s 179ms/step



Video URL:-

https://drive.google.com/file/d/1BqBKmvfV7sYco7PtMAZlhxNBifOJKB4C/view?usp=drive_link